

BEEMS: Boosting Machine Vision Efficiency via Computation Graph-Based Memory Smoothing

Hanjing Shen*
Shanghai Jiao Tong University,
Shanghai Qi Zhi Institute
Shanghai, China
shenhanjing@sjtu.edu.cn

Fangxin Liu*[†]
Shanghai Jiao Tong University,
Shanghai Qi Zhi Institute
Shanghai, China
liufangxin@sjtu.edu.cn

Jian Liu
Beihang University
Beijing, China
liujian@lynmaxtech.com

Li Jiang[†]
Shanghai Jiao Tong University,
Shanghai Qi Zhi Institute
Shanghai, China
ljiang_cs@sjtu.edu.cn

Haibing Guan
Shanghai Jiao Tong University
Shanghai, China
hbguan@sjtu.edu.cn

Abstract

With the rapid advances of deep learning-based computer vision (CV) technology, digital images are increasingly processed not by humans, but by downstream CV algorithms. In particular, the growing popularity of vision foundation models has heightened interest in deploying these models on edge devices. However, limited memory remains a key bottleneck, making memory footprint reduction essential. Mainstream model customization methods often require intensive deployment efforts and can severely degrade accuracy. Moreover, existing deep learning frameworks generally do not prioritize memory optimization. Existing memory management schemes face practical limitations, including layer-wise memory imbalance, high management overhead, and volatile memory budgets.

To tackle these issues, this work focuses on compilation-level optimizations that are explicitly designed to be memory-aware. We observe that memory usage during vision foundation model inference varies significantly over time (up to a 10× difference), with extended periods of low memory demand. Based on this, we propose BEEMS, a dual-objective compiler that optimizes both memory and latency by smoothing memory usage across the computational graph. BEEMS analyzes the vision foundation model computational graph to identify peak and trough operators in terms of memory demand. It then builds an efficient optimization search space, offering a flexible interface that applies different strategies

based on operator characteristics. Specifically, peak operators are optimized using techniques such as operator partitioning, kernel substitution, swapping, and rematerialization to reduce memory pressure, while trough operators apply subgraph substitutions to improve latency. Experiments on six diverse models show that BEEMS reduces peak memory by up to 90% and improves latency by 10%, demonstrating its effectiveness in jointly optimizing memory and performance.

CCS Concepts: • Computing methodologies → Machine learning; • Software and its engineering → Compilers.

Keywords: AI compilation, deep learning systems, memory optimization

ACM Reference Format:

Hanjing Shen, Fangxin Liu, Jian Liu, Li Jiang, and Haibing Guan. 2026. BEEMS: Boosting Machine Vision Efficiency via Computation Graph-Based Memory Smoothing. In *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '26)*, January 31 – February 4, 2026, Sydney, NSW, Australia. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3774934.3786430>

1 Introduction

In recent years, vision foundation models (VFM) have demonstrated remarkable achievements across diverse domains, including image recognition[9, 10], object detection and segmentation[2, 14], generative models such as Stable Diffusion [24]. As VFMs continue to deliver increasingly powerful performance, their architectures have grown substantially in both scale and complexity[14, 21, 24], leading to a dramatic rise in memory demands during inference. This escalating memory requirement has become a major bottleneck for on-device VFM deployment, making memory footprint reduction a critical challenge.

Optimizing memory usage is essential for both multi-tenant cloud environments and edge computing scenarios.

*Both authors contributed equally to the paper.

[†]Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.

PPoPP '26, Sydney, NSW, Australia

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2310-0/2026/01

<https://doi.org/10.1145/3774934.3786430>

In cloud settings, efficient memory utilization greatly improves hardware resource efficiency[20, 29]. For example, GPU servers (e.g., NVIDIA A100 with 40GB HBM2[7]) often host dozens of concurrent inference instances. Lowering memory consumption enables higher concurrency, reducing operational costs while sustaining throughput. In edge computing, where memory capacity and power are tightly constrained, minimizing memory usage is fundamental for deploying high-performance VFMs. Typical edge platforms include mobile SoCs (e.g., Qualcomm Snapdragon 8 series) with 4–8GB memory and embedded FPGAs with power budgets below 5W, which must still support models like ResNet-50 with sub-10ms inference latency.

The mainstream approach to deploying VFMs on memory-constrained devices is model compression, which applies techniques such as pruning, quantization, and knowledge distillation. However, these methods often sacrifice accuracy, prompting interest in alternatives that preserve the original structure. In compiler optimization, techniques like swapping, rematerialization, and operator partitioning have gained attention. They reduce peak memory by offloading and reloading intermediate results, recomputing activations, or breaking down large operators into smaller sub-operations. While effective at lowering memory usage, these methods introduce overhead—such as redundant computation from rematerialization and reduced parallelism from partitioning—which hampers hardware utilization.

On the other hand, several studies reduce latency via subgraph substitution, where predefined rules identify equivalent patterns and replace them with fused or parallelized operators (e.g., PyTorch, TVM). To expand coverage, automated search and relaxed substitution strategies have been explored, improving hardware utilization and reducing memory access overhead. However, these methods typically increase memory usage due to larger fused operators, making it difficult to balance latency gains with memory efficiency.

Balancing latency and memory in VFM inference is difficult for two main reasons: memory-oriented optimizations inevitably introduce redundant operations, and performance-driven subgraph substitutions often overlook memory impact in their evaluation, leading to higher peak memory usage. By analyzing memory profiles across operator execution sequences, we observe a significant gap (over 30%) between peak and average memory demands. Notably, 83% of operational phases use less than 70% of peak memory, revealing substantial potential for memory-aware optimization.

To address this challenge, we present BEEMS, a memory-aware computational graph optimization framework for vision foundation models (VFMs). The framework intelligently analyzes memory demand patterns during inference, identifying peak operators (memory-intensive) and valley operators (memory-light). It then applies hybrid strategies to different regions: implementing “peak shaving” to reduce memory pressure on critical operators, and “valley filling” to

adjust memory allocation for non-critical ones, enhancing computational efficiency. The framework consists of three key components: (1) a memory profiler that tracks runtime utilization through simulation; (2) an optimizer that dynamically selects strategies based on profiling results; and (3) a collection of techniques enabling fine-grained memory-performance tradeoffs across operator types.

In summary, our contributions are as follows:

- We identify a significant gap between peak and average memory demands during VFM inference. This reveals an untapped optimization potential and motivates memory-driven optimizations.
- We design a hybrid optimization strategy that systematically combines operator-level and subgraph-level techniques, enabling flexible tradeoffs between memory and performance. Our method extends the traditional search space by supporting heterogeneous optimization choices across different graph regions.
- We design a memory-aware compilation framework that combines runtime memory profiling, adaptive optimization selection, and tailored graph transformations, enabling fine-grained tradeoffs between memory and performance.
- Extensive experiments across six representative VFMs demonstrate that our approach reduces peak memory by up to 90% with minimal latency overhead, showcasing both its effectiveness and generalizability for practical deployment.

2 Background

2.1 DNN Inference Memory

Deep learning frameworks such as Caffe, TensorFlow, PyTorch, and ONNX represent neural networks as computational graphs—directed acyclic graphs (DAGs) where vertices correspond to operators (layers) and edges indicate tensor flow. For a computational graph $G = (V, E)$, executing a vertex involves: (1) ensuring all input tensors are in memory; (2) allocating memory for temporary workspaces (M_c) and output tensors (M_o); (3) performing the computation; and (4) releasing input tensors no longer needed by downstream operators.

Despite the widespread use of such frameworks, memory management remains a critical bottleneck, especially for on-device and edge deployments. Unlike computational cost—which primarily affects latency—memory is a hard resource constraint that directly determines whether inference can be executed at all. Hardware trends exacerbate this issue: while computational power (e.g., GPU FLOPS) has improved dramatically over the past decade, available memory capacity has grown at a much slower pace. Moreover, operating system policies (e.g., app-wise heap limits) and application-level constraints further restrict the usable memory footprint for DNN inference.

A key challenge lies in the temporal nature of tensor lifecycles. The lifetime of each tensor spans from allocation at its producer operator to deallocation after all its consumers have finished execution. During this time, tensors experience both an active period—when they are being produced or consumed—and an inactive phase, during which they occupy memory without any access. This mechanism, while simple and general-purpose, often results in substantial memory overhead, as tensors may persist in memory long after their active use has ended.

These observations highlight the need for memory-centric optimization strategies that go beyond purely computational improvements. By making memory a first-class optimization target, it becomes possible to bridge the growing gap between limited device memory and the increasing demands of modern DNN models, enabling more efficient and reliable deployment in constrained environments.

2.2 Graph Optimization

Modern AI compilers employ various optimizations to improve both latency and memory efficiency. Common methods include operator fusion, substitution, partitioning, swapping, and rematerialization. We survey these techniques in state-of-the-art frameworks, summarized in Table 1, with effectiveness and applicable operators detailed in Table 2.

Subgraph Substitutions. TASO [13] introduced subgraph substitution, which replaces a subgraph with a functionally equivalent alternative matching specific patterns. While early work focused on operator fusion, TASO generalized this to more complex subgraphs. This technique primarily reduces latency; some substitutions increase computational complexity to improve hardware utilization, often at the expense of higher memory usage.

Operator Partitioning. Cipolletta et al. [8] introduce a graph restructuring method that splits operators or subgraphs along specific dimensions to respect peak memory constraints. Each segment processes a portion of the input, repeating computation until the full output is produced. While this reduces per-segment memory usage, it can lower operator-level parallelism and degrade hardware utilization.

Swapping. Operator swapping (also called operator exchange) addresses GPU memory bottlenecks. It trades off memory usage and computational throughput by offloading inactive tensors from device memory (e.g., GPU VRAM) to host memory or storage. These tensors are reloaded when needed. Swapping enables larger models to run on devices with limited memory.

Rematerialization. Rematerialization (or recomputation) reduces peak memory consumption by recalculating intermediate tensors instead of storing them. Unlike traditional techniques such as common subexpression elimination (CSE),

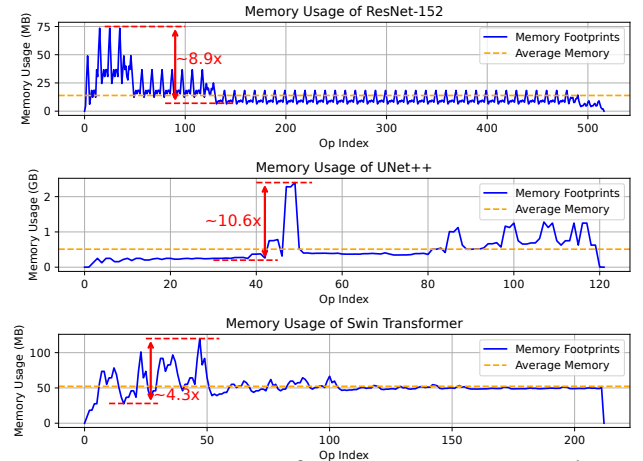


Figure 1. Memory usage of ResNet-152, UNet++, and Swin Transformer across operation indices, illustrating peak-to-average ratios: ResNet-152 ($\sim 8.9\times$), UNet++ ($\sim 10.6\times$), and Swin Transformer ($\sim 4.3\times$).

rematerialization traces operator dependencies and selectively regenerates critical tensors. This approach shortens tensor lifecycles through three steps: dependency tracing, cost-benefit analysis, and selective rematerialization.

3 Motivation

We perform a detailed memory profiling of VFM inference across both temporal and node-wise levels.

Observation #1: Unbalanced memory utilization across nodes. Figure 1 shows that VFM inference exhibits highly unbalanced memory usage over time. While peak memory determines the overall requirement, most nodes consume significantly less memory for extended periods. Specifically, the average memory utilization remains between 20% and 50%, with the lowest dropping to just 5%–15%. This gap between peak and average memory highlights a clear opportunity: many low-utilization nodes can be optimized through **subgraph substitutions** to improve performance. These nodes have ample memory headroom and can absorb additional overhead without increasing the peak memory footprint.

Key Insight 1 The large gap between peak and average memory usage allows safe optimization of low-utilization nodes (e.g., **subgraph substitutions**) to boost performance without violating peak memory constraints.

Observation #2: Sparse distribution of high-memory-utilization nodes. To further investigate memory bottlenecks, we analyze the distribution of memory demand across nodes. As shown in Figure 2, less than 10% of nodes consume more than 75% of peak memory, while the vast majority remain far below. This sparsity suggests that **memory-intensive optimizations** (such as operator partitioning and swapping) can be applied selectively to these few critical

Table 1. Summary of Graph Optimization Methods in AI Compilers

AI Compiler	Subgraph Substitutions	Operator Partitioning	Swapping	Rematerialization	Conv Kernel Substitution
TVM [5]	✓	✗	✗	✗	✗
DTR [15]	✗	✗	✗	✓	✗
DNNFusion [17]	✓	✗	✗	✗	✗
TASO [13]	✓	✗	✗	✗	✗
MAGIS [4]	✗	✓	✓	✓	✗
MoteNN [3]	✗	✓	✗	✗	✗
G10 [31]	✗	✗	✓	✗	✗
BEEMS(Ours)	✓	✓	✓	✓	✓

Table 2. Summary of Optimization technique

Optimization	Mem.	Lat.	Target Tensors
Subgraph Substitutions	▲	▼	Activate Tensors
Operator Partitioning	▼	▲	Activate Tensors
Swapping	▼	▲	Inactivate Tensors
Rematerialization	▼	▲	Inactivate Tensors

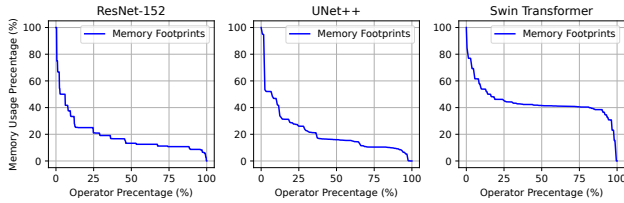


Figure 2. Sorted memory distribution of various models. Operator memory footprints are ranked (left to right) with normalized axes. In all models, the top 20% ~ 40% of operators contribute 80% ~ 90% of peak memory, while the remaining 60% ~ 80% occupy only 10% ~ 20%.

nodes. Because they are few, the associated performance overhead is minimal, enabling efficient memory reduction without significant impact on inference speed.

Key Insight 2 The sparsity of high-utilization nodes enables targeted use of memory-reduction techniques (e.g., **operator partitioning/swapping**) with minimal performance impact.

Observation #3: Potential conflicts between different optimization techniques. While different optimizations can enhance performance or reduce memory, naive combination often leads to conflicts. For example, as illustrated in Figure 3(a-b), subgraph substitutions may merge two matmul operators into a larger one to improve efficiency. However, this merged operator increases the memory footprint, creating a new peak. When operator partitioning is subsequently applied, the merged operator is divided back into smaller units, often using equal-sized partitions that introduce extra

split and concat overhead (Figure 3(c), where the operators marked with red boxes are redundant operators introduced by the naive combination of optimization). This leads to redundant computation and a suboptimal graph.

These cases illustrate that BEEMS is not a simple aggregation of existing techniques, such as those discussed in Section 2.2, and it is also not a trial-and-error approach that attempts to pick a seemingly good configuration. Instead, our design is driven by a key observation: when applied independently, optimizations that are individually beneficial can interfere with one another and significantly reduce overall effectiveness. This interference can produce suboptimal execution graphs, as shown in the **red-boxed** region in Figure 3, which incur higher latency or memory consumption. To this end, BEEMS explicitly separates conflicting transformations across peak and valley memory regions and adaptively selects the most suitable optimization based on operator-level memory behavior. This coordinated design ensures that each technique contributes positively without degrading overall performance.

Key Insight 3 Region-aware isolation prevents destructive interference between optimizations, enabling BEEMS to adaptively apply suitable transformations and avoid the degradation caused by naive combinations.

These observations highlight the need for a *region-aware optimization framework* that can: (1) exploit low-utilization regions for performance boosts; (2) selectively apply memory-saving techniques to sparse high-utilization nodes; and (3) isolate conflicting optimizations across regions to maintain efficiency and avoid redundancy.

4 BEEMS Framework

Figure 4 illustrates the overall architecture of the system. The framework comprises four key components: a memory emulator, joint optimization analysis, memory *peak shaving*, and memory *valley filling*. The system accepts a DNN computational graph (with ONNX-format input support) and outputs an optimized DNN graph with improved performance.

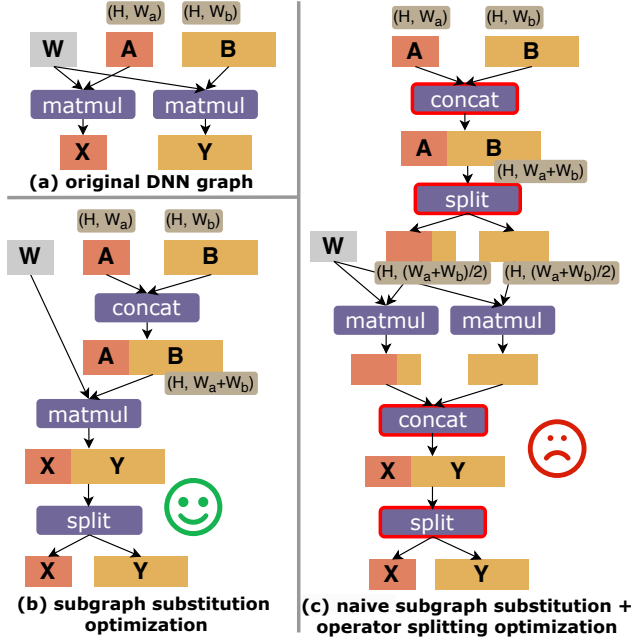


Figure 3. Comparison of computation graph optimizations. (a) Original graph with two sequential matrix multiplications ($W \otimes A$ and $W \otimes B$); (b) Optimized via subgraph substitutions, enhancing hardware utilization and parallelism; (c) Naive combination of subgraph substitutions and operator partitioning, resulting in fragmented operations (redundant splits, concatenations, and matmuls) that significantly increase graph complexity over (a).

4.1 Memory Profiling and Modeling

Memory profiling is critical for optimizing the memory efficiency of DNN systems. It involves analyzing memory usage patterns, identifying peak and valley demands, and applying strategies to minimize memory overhead while preserving computational performance. As illustrated in Figure 5, this process systematically captures tensor lifetime and memory dynamics across computational stages.

To obtain detailed memory usage during DNN inference, the network is first scheduled to acquire the operator execution sequence. A reverse post-order traversal is employed to determine the initial execution order. Next, the computational latency of each operator is measured. For single-stream execution, the execution timeline is constructed by aligning operators end-to-end. For multi-stream execution, operators are assigned to streams based on their execution order, and inter-stream dependencies are established according to operator dependencies. In BEEMS, computation and data movement are further decoupled by dispatching operator execution and tensor write-out/write-in operations onto separate streams, enabling effective overlap between computation and communication. The start time of each operator is determined by the maximum of (i) the end times of all its

dependent operators and (ii) the end time of the preceding operator in the same stream.

The lower section of Figure 5 explicitly shows the profiling outputs: The computation start and end times of each operator, the corresponding total memory demand, and detailed memory usage traces. Detailed usage traces are generated based on the DNN inference memory management model (Section 2.1), which records the allocation and deallocation times of each operator’s output tensors. All timelines are then aggregated to compute the total memory demand at each time point. At any given moment, the input and output tensors of the currently executing operator are considered *active*, while all others are *inactive*.

These active/inactive states, shown in the tensor lifetime panel, directly inform the optimization strategy selection described in Section 4.2.

4.2 Joint Memory and Performance Optimization

The goal of Joint Optimization is to iteratively improve memory efficiency and overall performance by orchestrating multiple optimization strategies within a unified framework. It leverages memory modeling results to pinpoint operators exhibiting peak or valley memory demands and analyzes their associated subgraph dependencies to guide targeted optimization actions.

The framework runs through an optimization manager that searches out various optimization graphs by invoking memory “peak shaving” and “valley filling” operations based on performance analysis feedback. After each optimization cycle, performance metrics are re-evaluated to decide whether and how to update the computation graph before the next iteration.

Algorithm 1 presents the details. The algorithm takes three primary inputs: (1) the computational graph to optimize, (2) user-defined latency constraint and (3) maximum iterations. It starts by building a prioritized candidate queue, which stores potential graph variants sorted by performance metrics. In each iteration, the highest-priority graph is dequeued for processing. The optimization manager then explores feasible optimization methods and generates distinct graph variants through systematic permutations.

BEEMS employs a hybrid exploration strategy: operator latency is empirically profiled once, while memory behavior is estimated through a lightweight simulator that tracks dynamic allocations during graph transformations. This design eliminates the need to execute every candidate graph variant, allowing BEEMS to accurately and efficiently evaluate both latency and memory impact. As a result, the system achieves low-overhead yet precise guidance during the search process, enabling rapid pruning of ineffective transformations.

To avoid redundant exploration of equivalent graph states, a hash-based verification mechanism is introduced. This

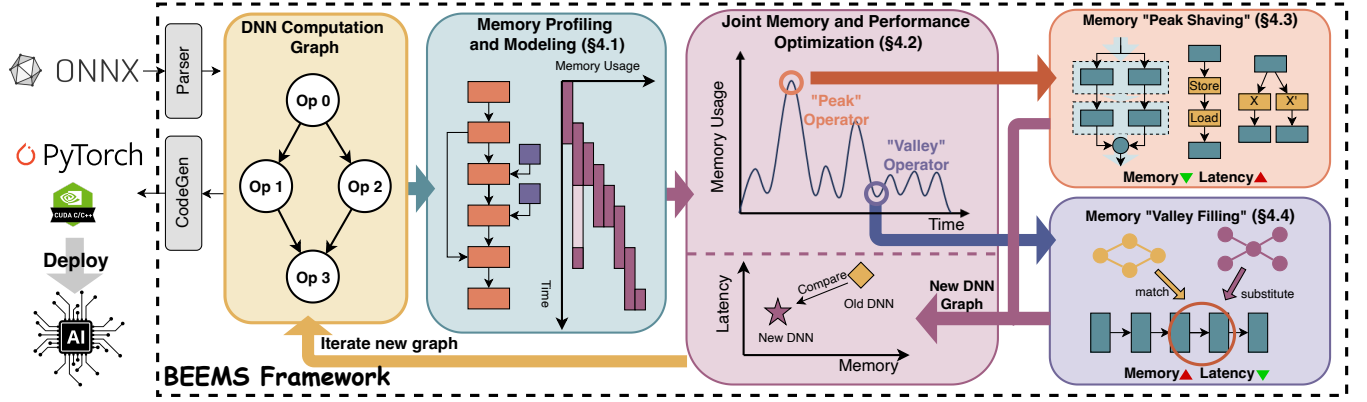


Figure 4. Overview of the BEEMS framework, comprising four key components: **memory emulator**, **joint optimization analysis**, **memory “peak shaving”**, and **memory “valley filling”**. The system optimizes a DNN computational graph by leveraging memory modeling, subgraph substitutions, and memory optimization strategies to enhance performance and reduce memory overhead.

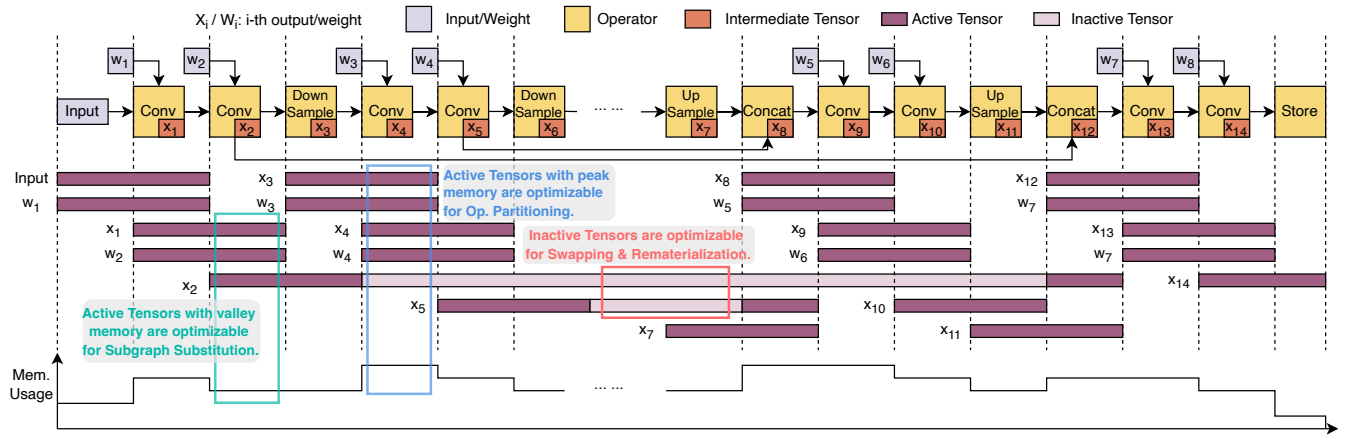


Figure 5. Memory lifecycle management in U-Net inference. The top workflow illustrates tensor state transitions through various operators. Inputs, weights, and intermediate tensors alternate between active and inactive states. Special optimizations are highlighted for different part of tensors. The bottom profiling shows the change in memory usage during inference.

mechanism computes a unique hash signature for each operator based on key features such as input/output tensor dimensions, operator attributes, and topological relationships. These are recursively composed to derive a network-level hash, enabling efficient isomorphism detection across different graph representations. This approach ensures diversity in the solution space while effectively eliminating redundant computations during the optimization process.

4.3 Optimizing Memory-Intensive Operators

To address memory bottlenecks caused by peak-memory operators, the BEEMS framework integrates four key techniques: **convolution kernel substitution**, **operator partitioning**, **swapping**, and **rematerialization**.

4.3.1 Kernel Substitution for Convolution. We propose a compilation-time optimization that substitutes convolution kernels based on their performance and memory profiles. Convolution operators can be implemented using various

kernels such as Winograd, FFT, and GEMM, each with distinct latency and workspace requirements. The workspace refers to GPU memory that must be allocated before execution and used during computation.

For example, we benchmarked different kernel implementations for specific convolution configurations on an A100 GPU using cuDNN. As shown in Figure 6, no single kernel consistently achieves optimal performance for both latency and memory consumption when the output feature map size remains constant. This observation motivates kernel substitution to optimize memory-critical workloads.

When a convolution operator causes the computational graph’s peak memory to exceed a predefined threshold, our algorithm applies the following procedure: it identifies all valid kernel candidates based on the operator’s characteristics and ranks them by latency. In each optimization iteration, the best-performing kernel is selected from the priority queue to generate a modified computation graph. The algorithm accepts the substitution only if the new kernel

Algorithm 1 Joint Optimization Algorithm

Require: Initial computation graph G_0 , target latency T_{max} , max iterations K

- 1: Profile G_0 to obtain latency and memory (L_0, M_0)
- 2: Initialize priority queue $Q \leftarrow \{(G_0, M_0)\}$ (sorted by M ascending)
- 3: $best_G \leftarrow G_0, best_M \leftarrow M_0, iteration \leftarrow 0$
- 4: **while** $iteration < K$ **and** $Q \neq \emptyset$ **do**
- 5: $(G_{current}, M_{current}) \leftarrow Q.pop_min()$
- 6: **if** $M_{current} < best_M$ **and** $L_{current} \leq T_{max}$ **then**
- 7: $best_G \leftarrow G_{current}, best_M \leftarrow M_{current}$
- 8: **end if**
- 9: Generate memory timeline $\mathcal{M}$ for $G_{current}$
- 10: Detect O_{peak} (peak ops), O_{valley} (valley ops)
- 11: $args \leftarrow (G_{current}, \mathcal{M}, O_{peak}, O_{valley})$
- 12: $G_{candidates} \leftarrow \text{Optimizer}(args)$
- 13: **for** $G_{new} \in G_{candidates}$ **do**
- 14: Profile (L_{new}, M_{new}) for G_{new}
- 15: **if** $L_{new} \leq T_{max}$ **and** $M_{new} < M_{current}$ **then**
- 16: $Q.insert(G_{new}, M_{new})$
- 17: **end if**
- 18: **end for**
- 19: $iteration \leftarrow iteration + 1$
- 20: **end while**
- 21: **return** $best_G$

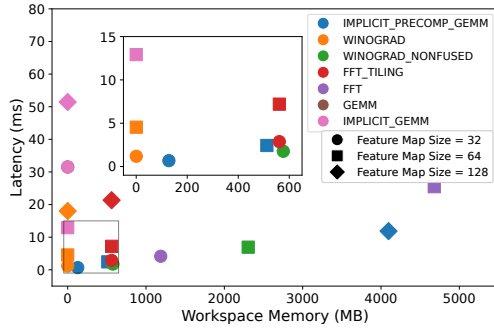


Figure 6. Performance trade-offs of 2D convolution algorithms: Latency (ms) vs. Workspace Memory (MB) across different algorithms and feature map sizes. As no algorithm minimizes both metrics simultaneously, BEEMS dynamically selects kernels to balance latency and memory usage.

improves at least one metric (latency or memory usage) without degrading the other. If no kernel meets this criterion, the modification is rejected and the iteration ends.

4.3.2 Operator Partitioning. Operator partitioning decomposes a large operator into multiple sub-operators. The process involves: (1) identifying the subgraph region to be split, (2) determining partitioning points and the number of subgraphs, and (3) applying the split.

Because operator connections are constrained by i/o dimensions, partitioning peak-memory operators also requires co-partitioning their upstream and downstream operators. We define the computational graph formed by these interconnected operators as the target subgraph. To facilitate partitioning, we adopt the Dimension Graph concept from MAGIS [4]. When the subgraph contains weight-bearing operators (e.g., convolutions), we include their weight tensors to ensure correct partitioning.

To model the effect of partitioning, we decompose the original peak memory into three components: I , the input tensors and any non-partitionable intermediates that must remain resident; W , the operator workspace; and M_o , the full output size, such that $M_p = I + W + M_o$. When splitting a dimension of length d into n segments, the workspace typically scales to W/n , while I remains fully allocated during execution. The output is produced in n fragments of size M_o/n , and merging these fragments introduces additional temporary buffering that depends on the merge strategy. The post-partition peak memory can therefore be estimated by:

$$M' \approx \max \left\{ I + \frac{W}{n} + M_o, I + c_{\text{red}} \frac{M_o}{n}, I + M_o + O_{\text{trans}} \right\}.$$

This upper bound captures the three dominant contributors to post-partition memory: the reduced workspace combined with the full output buffer, the temporary buffers required during reduction, and the allocation needed for concatenation and any layout adjustments. The largest of these terms determines the effective peak memory.

After partitioning, the subgraph is encapsulated as a partitioned operator. Scheduling continues using reverse post-order (RPO) traversal: when a partitioned subgraph is encountered, its internal operators are scheduled following RPO and executed n times (where n is the partitioning factor). The output tensors of subgraph remain allocated until all partitions complete.

Two cases arise for handling output tensors:

1. **Reduction Case:** When n partial outputs require reduction (e.g., summation), each fragment has size M_o/n . We allocate two buffers (a and b) of this size. The first segment is written to a, and the second to b; they are then reduced, with the result overwriting one of the buffers. For subsequent segments, each new fragment is written to the free buffer, reduced with the accumulator, and the result overwrites the other buffer, alternating until all n fragments are merged. This corresponds to the $c_{\text{red}} \frac{M_o}{n}$ ($c_{\text{red}} = 2$) term in the memory bound.
2. **Concatenation Case:** When n partial outputs are concatenated, we allocate a contiguous block of size M_o and write each fragment sequentially. If the partitioning dimension aligns with the outermost layout, no

extra steps are needed; otherwise, a transpose operator is inserted after execution to correct the layout, represented by the O_{trans} term in the bound.

4.3.3 Swapping and Rematerialization. Both swapping and rematerialization target *inactive tensors*, and BEEMS integrates them via a unified interface.

Swapping offloads activation tensors after computation and prefetches them when needed. In our implementation, data is transferred between GPU and CPU memory.

Rematerialization applies when an operator V has multiple successors (e.g., A , B). Instead of retaining V 's output in memory, the system recomputes V before each successor executes, reducing peak memory.

Algorithm 2 describes the procedure. Given a computation graph G and its memory usage trace (`mem_usage`), the algorithm calls `SortInactivePeakTensors` (line 14) to identify and score inactive tensors based on their size and inactive duration (line 6). Candidates are sorted in descending order to prioritize impactful optimizations. For each tensor, the algorithm emits a `SwapOut` directive and conditionally a `Rematerialize` directive if the tensor has multiple successors, enabling rematerialization to further reduce memory.

Algorithm 2 Swapping & Rematerialization

```

1: function SORTINACTIVEPEAKTENSORS(mem_usage)
2:    $V \leftarrow \text{MemoryPeakTensors}(\text{mem\_usage})$   $\triangleright$  Find
   tensors at memory peak
3:    $C \leftarrow \emptyset$ 
4:   for  $v \in V$  do
5:     if IsInactive( $v$ ) then
6:        $\text{score} \leftarrow \text{MemorySize}(v) \times \text{InactiveTime}(v)$ 
7:        $C \leftarrow C \cup \{(v, \text{score})\}$ 
8:     end if
9:   end for
10:   $C_{\text{sorted}} \leftarrow \text{SortByScoreDescending}(C)$ 
11:  return  $[v \mid (v, \_) \in C_{\text{sorted}}]$ 
12: end function
13: procedure GENERATESWAPREMATPLAN( $G, \text{mem\_usage}$ )
14:   $\text{args} \leftarrow \text{mem\_usage}$ 
15:   $\text{candidates} \leftarrow \text{SortInactivePeakTensors}(\text{args})$ 
16:   $\text{Plan} \leftarrow \emptyset$ 
17:  for  $t \in \text{candidates}$  do
18:     $\text{Plan} \leftarrow \text{Plan} \cup \{\text{SwapOut}(t)\}$ 
19:    if  $|\text{Successors}(t, G)| > 1$  then
20:       $\text{Plan} \leftarrow \text{Plan} \cup \{\text{Rematerialize}(t)\}$ 
21:    end if
22:  end for
23:  return  $\text{Plan}$ 
24: end procedure

```

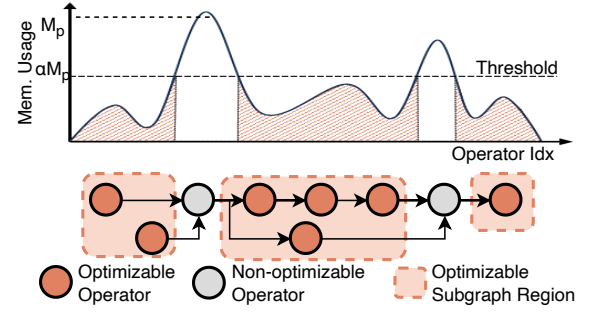


Figure 7. Identification of “valley operator” regions based on memory usage. The plot shows operator indices versus memory consumption, with operators below a predefined threshold (dotted line) marked as valley operators. These form contiguous, optimizable subgraph regions, while high-memory operators remain unmodified.

4.4 Optimization of Memory Valley Operators

Memory valley optimization targets underutilized memory periods during DNN inference, where available memory resources exceed immediate computational demands. BEEMS exploits these valleys through **subgraph substitutions**, strategically increasing memory utilization while improving throughput. The core approach of subgraph substitutions involves searching for subgraphs with equivalent topological structures within a neural network, based on a subgraph substitutions library. After evaluation, a decision is made on whether to perform the substitution. We utilize a substitution library generated by the TASO [13] generator.

We adopt a memory-aware, perception-guided subgraph substitutions approach. Figure 7 shows a method for identifying subgraph regions that can be optimized. For a DNN model with peak memory consumption M_p , we propose a valley threshold defined as αM_p , where regions below this threshold are designated as memory valleys. The parameter α is not fixed; instead, it adapts according to the target memory constraint, increasing when the system permits more temporary memory usage and decreasing under tighter memory budgets. This adaptive setting ensures that the optimized network never exceeds the target memory constraint. Within the valleys, the algorithm performs topological structure matching to identify candidate subgraphs. Upon successful matching, a performance simulation is conducted. The substitution is only executed when the simulation demonstrates measurable performance improvement, thereby guaranteeing optimization efficiency.

This optimization exploits idle memory bandwidth by increasing temporary memory usage, thereby improving overall throughput without impacting peak memory. The algorithm ensures that while optimizing for throughput, the peak memory utilization remains unchanged, thus preserving the original memory budget.

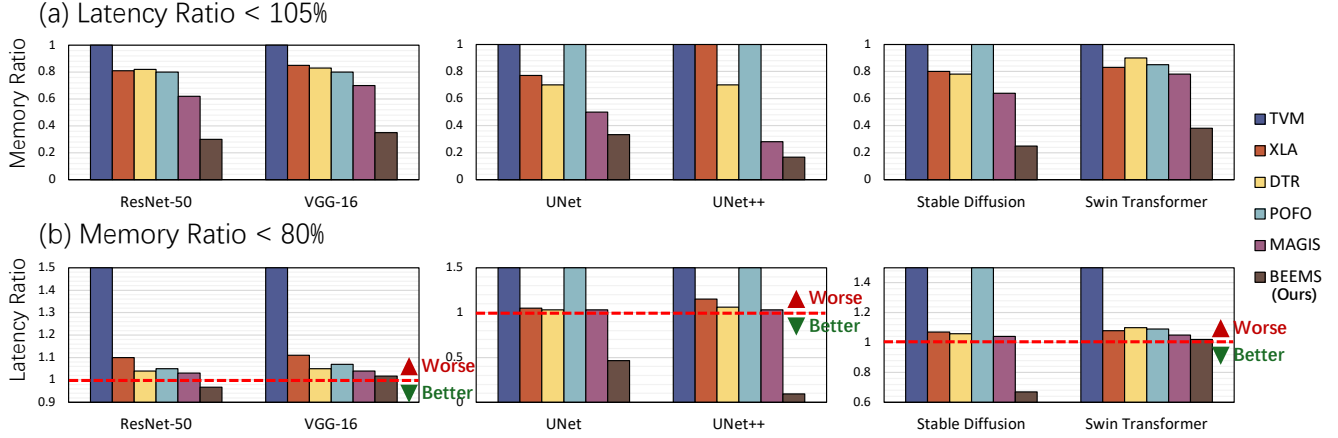


Figure 8. Comparison of the BEEMS framework with baseline methods (TVM, XLA, DTR, POFO, MAGIS) across multiple neural networks (ResNet-50, VGG-16, UNet, UNet++, Stable Diffusion, Swin Transformer). (a) Memory ratio under constrained latency (latency ratio < 105%); (b) Latency ratio under constrained memory (memory ratio < 80%). BEEMS demonstrates superior efficiency in balancing memory and latency trade-offs, outperforming other methods in most cases.

Table 3. Batch size and input shapes for evaluated models.

Model	Batch Size	Input Shape
ResNet-50[10]	64	[3, 224, 224]
VGG-16[28]	64	[3, 224, 224]
UNet[25]	32	[1, 256, 256]
UNet++[34]	32	[1, 256, 256]
Stable Diffusion[24]	64	[3, 512, 512]
Swin Transformer[16]	64	[3, 224, 224]

5 Experiment

5.1 Experimental Methodology

Implementation. The BEEMS framework is designed and implemented as an extensible AI computational graph compiler architecture, fully developed in Python. Its IR adopts a graph-based representation, where each operator is defined as a vertex. To extend support for new operators, users need to: (1) define the operator’s attribute list, (2) implement parser compatibility, and (3) add target hardware-specific codegen implementations. Typically, extending an operator requires around one hour of expert-level development effort.

The architecture consists of modular components, including a parser, optimizer, and codegen, all designed for straightforward extensibility. The parser currently supports ONNX input formats but can be extended to other frameworks such as PyTorch. The codegen module generates PyTorch code, with convolution operations specifically optimized using cuDNN kernels. Although the current target is NVIDIA GPUs, the framework can be cost-effectively ported to other hardware platforms by extending the codegen module. Optimization strategies are orchestrated via an optimization controller, with peak-valley optimization built on top of TASO.

Additional optimization techniques can be integrated via this controller.

Platform. Experiments run on a server with dual AMD EPYC 7542 CPUs (32 cores/socket) and an NVIDIA A100 80GB GPU using CUDA 12.4. Results are averaged over 5 runs with identical hardware settings. The software environment is as follows: PyTorch version 2.4.0, cuDNN version 9.3.0, ONNX version 1.17.0, and Protobuf version 4.25.3.

Network. Table 3 summarizes the batch size and input resolutions used for each network in our experiments. We selected the following networks as experimental workloads: (a) ResNet [10], a CNN with residual connections; (b) VGG [28], a CNN built by stacking multiple 3×3 convolution layers with max pooling; (c) UNet [25] and its variant UNet++ [34], both using skip connections for medical image segmentation; (d) Stable Diffusion [24], A generative model; and (e) Swin Transformer [16], a vision transformer with shifted windows. All networks are tested with FP32 precision.

Baseline. We compare against several state-of-the-art compilers: (a) TVM [5], supporting memory management and operator fusion; (b) XLA [26], using a greedy memory scheduler; (c) DTR [15], which uses rematerialization for memory management; (d) POFO [1], combining rematerialization and swapping; and (e) MAGIS [4], a training-focused memory optimization compiler that integrates fission, swapping, and rematerialization. These represent the most influential approaches in memory-aware compiler optimization.

5.2 Experimental Results

5.2.1 Memory Optimization with Latency Constraint.

We first evaluate BEEMS under a latency constraint of < 1.05× the original runtime (Figure 8(a)). The memory ratio denotes optimized memory usage relative to the original graph. Under this constraint, BEEMS reduces memory by 15%–40%,

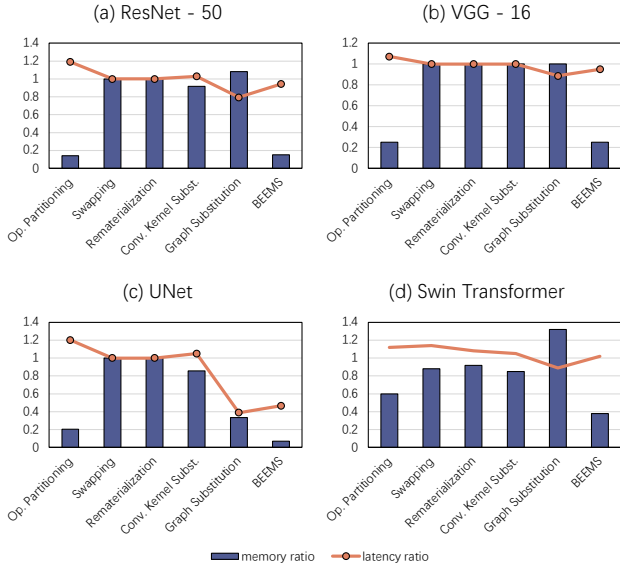


Figure 9. Comparison of memory and latency optimization across different methods for (a) ResNet-50, (b) VGG-16, (c) UNet and (d) Swin Transformer.

compared to MAGIS (25%–85%), POFO (80%–100%), DTR (70%–90%), XLA (75%–100%), and TVM ($\approx 100\%$). On CNNs such as ResNet-50 and VGG-16, BEEMS achieves 15%–33% savings, leveraging the large peak–average gap caused by wide intermediate channels. On UNet, memory drops by 17%–33% thanks to both CNN-like patterns and richer sub-graph structures. On Swin Transformer, memory decreases to 40% of the original.

5.2.2 Latency Optimization with Memory Constraint. We then evaluate latency under a memory constraint of $< 80\%$ (Figure 8(b)). The y-axis shows latency ratio (optimized/original), where bars below 1.0 indicate speedup. BEEMS maintains latency within 10%–107%, consistently outperforming baselines. POFO and MAGIS offer moderate gains (102%–115%) on ResNet-50 and Stable Diffusion but degrade up to 150% on UNet-like models. TVM, XLA, and DTR generally increase latency beyond 110%. These results highlight its suitability for memory-constrained, high-efficiency inference workloads.

5.2.3 Ablation Study. Figure 9 shows the effect of individual optimizations. Operator partitioning consistently reduces memory, while swapping, rematerialization, and kernel substitution offer limited benefits. Subgraph substitutions improve latency but often incur large memory penalties. BEEMS balances these trade-offs: on UNet, it combines subgraph substitutions with peak-shaving to reduce both memory and latency, while on Swin Transformer it avoids the memory overhead of subgraph substitutions, achieving a 38% memory ratio with only 2% latency increase.

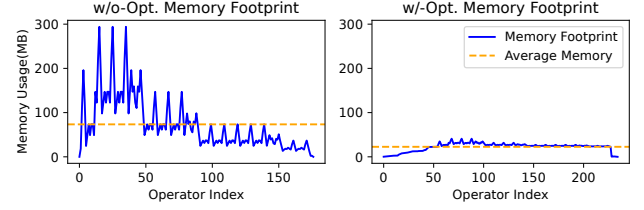


Figure 10. Comparison of memory footprints without optimization and with optimization for ResNet-50, showing memory usage and average memory across operator indices.

5.2.4 Breakdown of Memory Footprint. Figure 10 shows the memory profile of ResNet-50 before and after optimization based on peak–valley analysis with BEEMS. The original profile exhibits high volatility, with peaks $\sim 4\times$ the average and $10\times$ the valleys. After optimization, the peak-to-average ratio drops to $1.4\times$, and 75% of operators run above the average memory level, indicating well-balanced allocation.

Table 4. Latency Breakdown Analysis of ResNet-50 (Batch Size=128) With Memory Optimization Ratio = 15%.

Operator Type	w/o-Opt. (ms)	w-Opt. (ms)
Convolution 2D	24.50	29.78
ReLU	8.80	5.07
Pooling 2D	1.36	0.98
Element-wise Add	7.61	3.97
Total	42.27	39.79

5.2.5 Breakdown of Operator Latency. Table 4 reports operator latencies under a 15% memory reduction. Overall latency improves from 42.27 ms to 39.79 ms. ReLU and Element-wise Add show the largest gains (reducing from 8.80→5.07 ms and 7.61→3.97 ms), while Convolution 2D incurs overhead (24.5→29.8 ms) due to operator partitioning and kernel substitution. Despite such cases, the net effect remains positive, confirming that BEEMS effectively balances memory savings with latency reduction.

5.2.6 Sensitivity Analysis.

Batch Size. Figure 12 illustrates the effect of batch size under a 110% latency constraint. On UNet, memory usage drops below 10% even at medium batch sizes while latency remains 40%–50%, demonstrating strong benefits from larger batches. ResNet-50 shows a similar trend, reaching 15% memory ratio at batch size 128. Other models follow the same pattern with varying magnitudes: CNNs benefit significantly, while Transformer models remain stable with nearly constant latency. Overall, these results confirm that BEEMS scales effectively with batch size, amplifying memory savings without violating latency constraints.

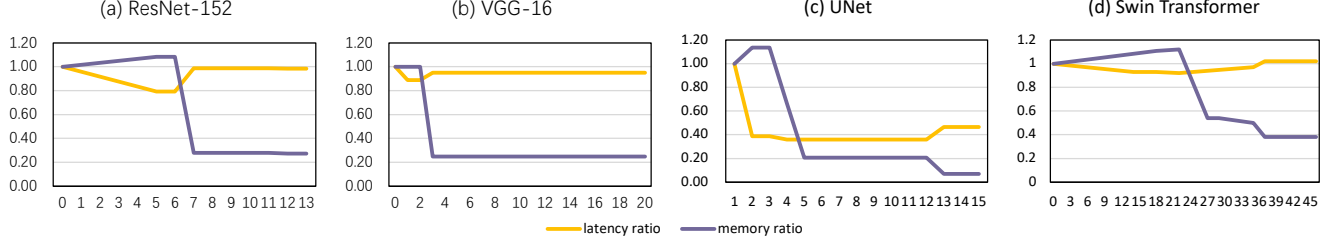


Figure 11. Evolution of latency ratio and memory ratio across iterations for four network architectures.

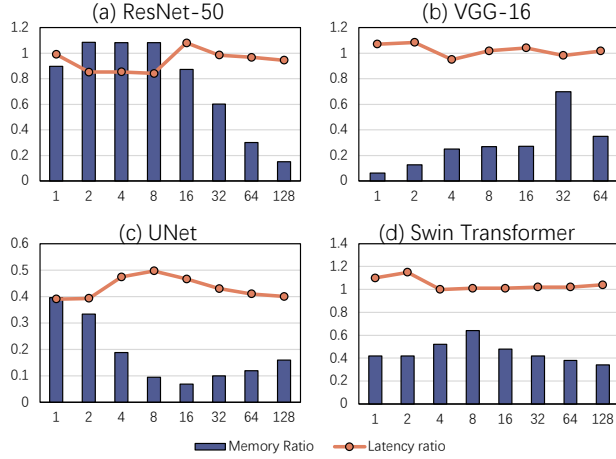


Figure 12. Latency and Memory Ratio for ResNet-50, VGG-16, UNet and Swin Transformer with various batch size.

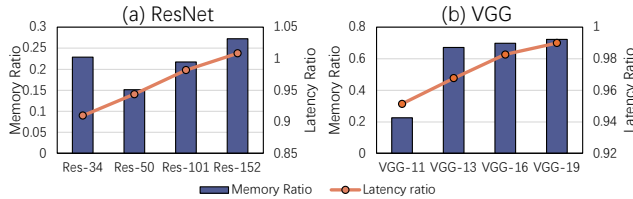


Figure 13. Comparison of (a) ResNet and (b) VGG, illustrating memory ratio (purple bars, left axis) and latency ratio (orange line, right axis) across increasing model depths.

Iteration Times. Figure 11 shows the convergence of BEEMS across different models. In all cases, the optimization stabilizes within 40 iterations, though the exact convergence speed varies by model. Two consistent patterns emerge: (1) the memory ratio initially rises before dropping, while (2) the latency ratio decreases rapidly and then slightly increases. This two-phase behavior reflects the decoupled nature of the optimization—early iterations reduce latency by targeting valley operators, while later iterations focus on memory reduction at peak operators. Overall, this iterative process is efficient in practice, with the end-to-end compilation completing in approximately 3-5 minutes, which remains well within an acceptable range.

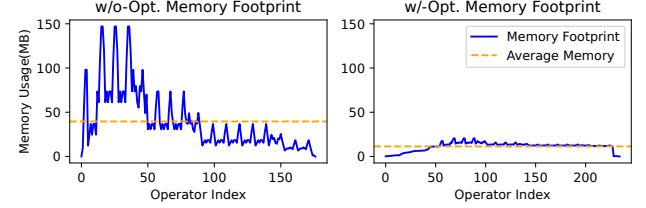


Figure 14. Comparison of memory footprints without optimization and with optimization for ResNet-50 under FP16 precision. The optimization significantly smooths memory fluctuations and reduces peak usage.

Model Scalability Analysis. Figure 13 evaluates BEEMS on multiple ResNet (34–152) and VGG (11–19) variants under a 110% latency constraint. Across the ResNet family, BEEMS reduces memory by 15%–28% with latency ratios between 91%–101%. Optimization is generally more effective on smaller models (e.g., ResNet-34, VGG-11) than on deeper ones (e.g., ResNet-152, VGG-19), reflecting diminishing returns as model depth increases. This trend arises because (1) the fraction of optimizable operators decreases as network depth grows, and (2) the peak-to-average memory gap narrows with depth, limiting headroom for transformation.

Model Precision. Figure 14 illustrates the memory footprint of ResNet-50 under FP16 precision before and after applying BEEMS. Due to the changes in operator execution time and data-transfer latency introduced by half-precision tensors, the pre-optimization memory trace differs noticeably from its FP32 counterpart. Nevertheless, the optimized execution still achieves a substantial reduction in peak memory. This demonstrates that BEEMS remains effective and robust across different numerical precisions, consistently lowering memory peaks even when computation–communication dynamics shift under FP16.

6 Related Work

Operator Partitioning. Operator partitioning decomposes large operators into smaller sub-operators to avoid memory overflow. Originating from operator tiling [27], partitioning works at the graph level, whereas tiling operates at

the compiler's operator level. Previous work includes Cipolletta et al. [8], who controlled memory via constrained operator widths, and MAGIS [4], which used Fission Transformation for partitioning. Unlike static methods, BEEMS applies memory-aware techniques to automatically determine partitioning boundaries and granularity.

Swapping. Swapping extends memory by offloading tensors to external storage and reloading on demand. Early systems like vDNN [23] and Capuchin [19] demonstrated their effectiveness, with later works introducing sophisticated methods such as SwapAdvisor [11], which co-optimized tensor reordering and swapping. POET [18] combined rematerialization with swapping, and ZeRO-Offload [22] extended this to distributed training.

Rematerialization. Rematerialization reduces memory by evicting intermediate tensors and recomputing them when needed. Initially introduced in [6], subsequent works like DTR [15] focused on optimizing this technique for dynamic graphs.

Subgraph Substitutions. Subgraph substitutions, originating from superoptimization techniques, improves operator fusion. TVM [5] was an early example, and later works like APOLLO [32] integrated memory optimization with operator fusion. TASO [13] expanded fusion rules, while GSPO [30] integrated substitution with distributed computing.

Other systems, such as AutoGraph [33] and WiseGraph [12], optimize at the graph level, targeting operator parallelism and Graph Neural Networks (GNNs), respectively. However, BEEMS goes further by incorporating advanced memory management techniques into the graph-level optimizations, ensuring that both memory and performance are optimized within a unified framework.

7 Conclusion

We present BEEMS, a novel DNN graph compiler that optimizes memory efficiency while maintaining controlled computational latency focusing on vision foundation models. Our framework uniquely addresses both peak-memory and valley-memory operators through integrated graph optimization techniques, achieving effective memory smoothing without compromising inference performance. BEEMS demonstrates remarkable memory reduction (15% to 40%) with minimal latency overhead.

Acknowledgements

This work was partially supported by the National Key Research and Development Program of China (2024YFE0204300), the National Natural Science Foundation of China (Grant No. 62402311), and the Natural Science Foundation of Shanghai (Grant No. 24ZR1433700).

References

- [1] Olivier Beaumont, Lionel Eyraud-Dubois, and Alena Shilova. 2021. Efficient combination of rematerialization and offloading for training

- dnns. *Advances in Neural Information Processing Systems* 34 (2021), 23844–23857.
- [2] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. End-to-end object detection with transformers. In *European conference on computer vision*. Springer, 213–229.
- [3] Renze Chen, Zijian Ding, Size Zheng, Meng Li, and Yun Liang. 2024. MoteNN: Memory Optimization via Fine-grained Scheduling for Deep Neural Networks on Tiny Devices. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [4] Renze Chen, Zijian Ding, Size Zheng, Chengrui Zhang, Jingwen Leng, Xuanzhe Liu, and Yun Liang. 2024. Magis: Memory optimization via coordinated graph transformation and scheduling for dnn. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 607–621.
- [5] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.
- [6] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174* (2016).
- [7] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. Nvidia a100 tensor core gpu: Performance and innovation. *IEEE Micro* 41, 2 (2021), 29–35.
- [8] Antonio Cipolletta and Andrea Calimera. 2021. On the efficiency of sparse-tiled tensor graph processing for low memory usage. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 643–648.
- [9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [11] Chien-Chin Huang, Gu Jin, and Jinyang Li. 2020. Swapadvisor: Pushing deep learning beyond the gpu memory limit via smart swapping. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 1341–1355.
- [12] Kezhao Huang, Jidong Zhai, Liyan Zheng, Haojie Wang, Yuyang Jin, Qihao Zhang, Runqing Zhang, Zhen Zheng, Youngmin Yi, and Xipeng Shen. 2024. WiseGraph: Optimizing GNN with joint workload partition of graph and operations. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 1–17.
- [13] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*. 47–62.
- [14] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. 2023. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*. 4015–4026.
- [15] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. 2020. Dynamic tensor rematerialization. *arXiv preprint arXiv:2006.09616* (2020).
- [16] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*. 10012–10022.
- [17] Wei Niu, Jiexiong Guan, Yanzhi Wang, Gagan Agrawal, and Bin Ren. 2021. Dnnfusion: accelerating deep neural networks execution with

- advanced operator fusion. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 883–898.
- [18] Shishir G Patil, Paras Jain, Prabal Dutta, Ion Stoica, and Joseph Gonzalez. 2022. POET: Training neural networks on tiny devices with integrated rematerialization and paging. In *International Conference on Machine Learning*. PMLR, 17573–17583.
 - [19] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. 2020. Capuchin: Tensor-based gpu memory management for deep learning. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 891–905.
 - [20] B Pratheek, Neha Jawalkar, and Arkaprava Basu. 2021. Improving gpu multi-tenancy with page walk stealing. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 626–639.
 - [21] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PMLR, 8748–8763.
 - [22] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. {Zero-offload}: Democratizing {billion-scale} model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 551–564.
 - [23] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W Keckler. 2016. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1–13.
 - [24] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2022. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10684–10695.
 - [25] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*. Springer, 234–241.
 - [26] Amit Sabne. 2020. Xla: Compiling machine learning for peak performance. *Google Res* (2020).
 - [27] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. 2023. Welder: Scheduling deep learning memory access via tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. 701–718.
 - [28] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
 - [29] Kan Wu, Kaiwei Tu, Yuvraj Patel, Rathijit Sen, Kwanghyun Park, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2022. {NyxCache}: Flexible and efficient multi-tenant persistent memory caching. In *20th USENIX Conference on File and Storage Technologies (FAST 22)*. 1–16.
 - [30] Zheng Xu, Xu Dai, Shaojun Wei, Shouyi Yin, and Yang Hu. 2024. GSPO: A Graph Substitution and Parallelization Joint Optimization Framework for DNN Inference. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
 - [31] Haoyang Zhang, Yirui Zhou, Yuqi Xue, Yiqi Liu, and Jian Huang. 2023. G10: Enabling an efficient unified gpu memory and storage architecture with smart tensor migrations. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 395–410.
 - [32] Jie Zhao, Xiong Gao, Ruijie Xia, Zhaochuang Zhang, Deshi Chen, Lei Chen, Renwei Zhang, Zhen Geng, Bin Cheng, and Xuefeng Jin. 2022. Apollo: Automatic partition-based operator fusion through layer by layer optimization. *Proceedings of Machine Learning and Systems 4* (2022), 1–19.
 - [33] Yuxuan Zhao, Qi Sun, Zhuolun He, Yang Bai, and Bei Yu. 2023. Auto-Graph: Optimizing DNN computation graph for parallel GPU kernel execution. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 11354–11362.
 - [34] Zongwei Zhou, Md Mahfuzur Rahman Siddiquee, Nima Tajbakhsh, and Jianming Liang. 2018. Unet++: A nested u-net architecture for medical image segmentation. In *Deep learning in medical image analysis and multimodal learning for clinical decision support: 4th international workshop, DLMIA 2018, and 8th international workshop, ML-CDS 2018, held in conjunction with MICCAI 2018, Granada, Spain, September 20, 2018, proceedings 4*. Springer, 3–11.

Received 2025-08-25; accepted 2025-11-10